

Desarrollo de Autenticación y Acceso al Sistema

Actividad N°: 17 **Fecha:** 23/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

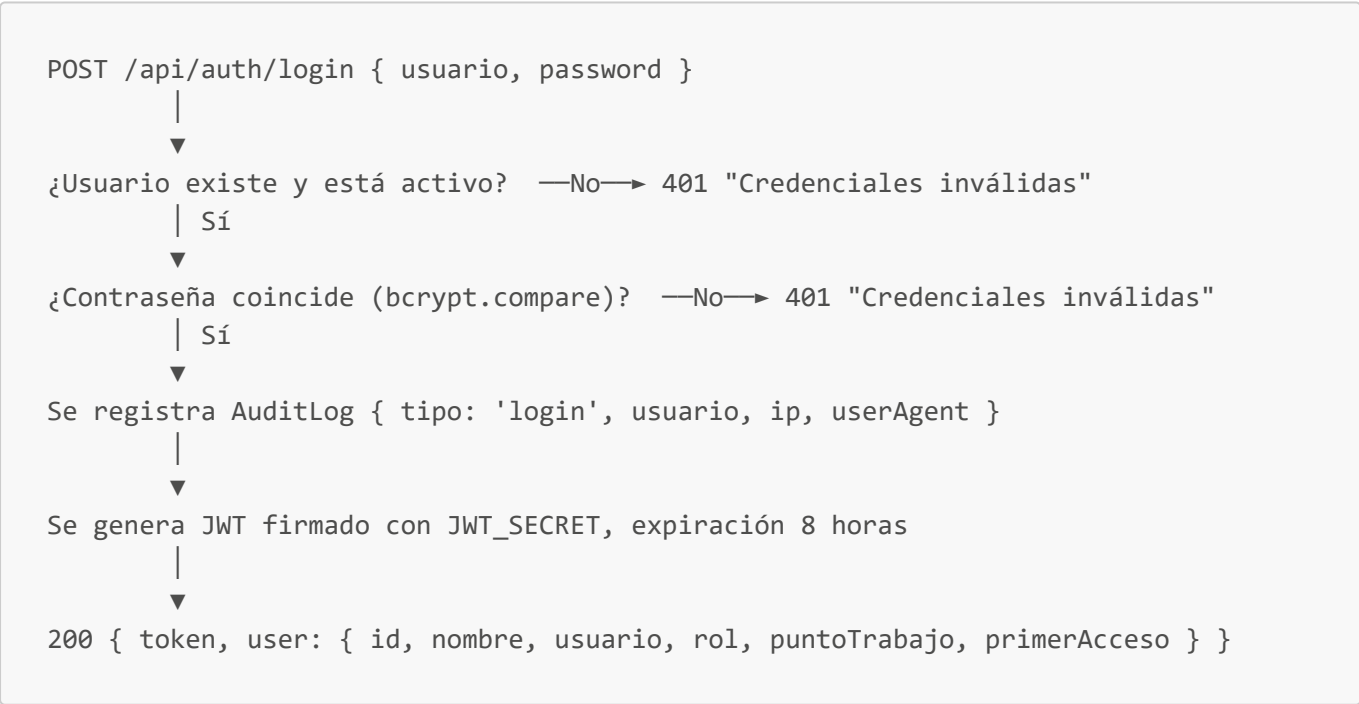
1. Objetivo del día

Implementar y documentar el módulo de autenticación: inicio de sesión, generación y verificación de token, cambio de contraseña en primer acceso, y cierre de sesión.

2. Endpoints implementados (src/routes/auth.js + authController.js)

Método	Ruta	Descripción
POST	/api/auth/login	Autentica usuario y devuelve token JWT
POST	/api/auth/change-password	Cambia la contraseña (requiere sesión)
POST	/api/auth/logout	Registra el cierre de sesión en auditoría
GET	/api/auth/profile	Devuelve los datos del usuario autenticado

3. Flujo de login implementado



Decisión de diseño relevante: el mensaje de error es idéntico ("Credenciales inválidas") tanto si el usuario no existe como si la contraseña es incorrecta, evitando que un atacante pueda enumerar usuarios válidos por diferencia de respuesta.

4. Flujo de cambio de contraseña (incluye primer acceso)

```
POST /api/auth/change-password { currentPassword?, newPassword }
    |
    ▼
¿primerAcceso === true?
├─ Sí → no exige currentPassword (primer cambio obligatorio)
├─ No → exige y valida currentPassword contra el hash almacenado
    |
    ▼
Nueva contraseña ≥ 6 caracteres → se reasigna (hook pre-save la hashea)
    |
    ▼
primerAcceso pasa a false
    |
    ▼
Se registra AuditLog { tipo: 'cambio_password' }
```

Esto asegura que todo usuario nuevo (creado con la contraseña por defecto `DEFAULT_PASSWORD`) esté obligado a definir una contraseña propia antes de operar con normalidad.

5. Middleware de verificación de sesión (`middleware/auth.js`)

- Extrae el token del header `Authorization: Bearer <token>`.
- Verifica la firma y expiración con `jwt.verify`.
- Recupera el usuario en base de datos (excluyendo `password`) y valida que `activo === true`.
- Adjunta `req.user` para que esté disponible en controladores y en los middlewares de autorización/auditoría siguientes.

6. Frontend — `AuthContext`

- Centraliza el estado de sesión (usuario, token) y expone `login()`, `logout()`, `isAuthenticated`.
- Persiste el token para mantener la sesión entre recargas de página.
- Tras un login exitoso, `RoleBasedRedirect` decide la pantalla de destino según el rol (`jefe` → `/dashboard`, `staff` → `/tickets`).

7. Conclusiones del día

El módulo de autenticación queda funcional de extremo a extremo: login seguro sin filtración de información sobre qué credencial falló, manejo obligatorio de contraseña en primer acceso, verificación de sesión vía JWT en cada request protegido, y trazabilidad de login/logout en auditoría.

Observaciones: Sin observaciones.